
A comparative analysis of Von-Heijne, SVM and Perceptron Models for the detection of Signal Peptides.

Fabrizio Guidotti¹

¹Department of Pharmacy and Biotechnology, University of Bologna, Italy

Abstract:

Motivation: Signal peptides enable proteins to enter the secretory pathway. Given their importance and the high cost of in-vivo assays, accurate in-silico SP detection models have taken on an important role in bioinformatics.

Results: With this paper, we provide two models for the detection of Signal Peptides from sequence information. The first model, based on the Von-Heijne algorithm, produces a relatively accurate model, with an MCC of 0.66. The second model is based on Support Vector Machines and achieves an higher MCC of 0.86 and f_1 score of 0.87, supporting an overall stronger model. Given the complexity of the full SVM model, we also trained a reduced model, which trades some performance (MCC of 0.73, and f_1 of 0.76), for a significantly lower number of features necessary, lowering the amount of preprocessing needed and the overall complexity of the final model.

1. Introduction

For a protein to enter the secretory pathway, in both eukaryotic and prokaryotic cells, it must be endowed with a specific target signal. Often, this signal takes the shape of a short sequences located at the N-terminus of proteins [1]. The signal peptide (SP) has a distinct three-domain structure, depicted in Figure 1, with a positively charged N-terminal region (n-region), a central hydrophobic region (H-region) and a more polar C-terminal region (C-region) containing the cleavage site [1]. Given their importance in many aspects of cell biology, the accurate detection of SPs is a crucial task in bioinformatics, which has been tackled before with a variety of approaches, including machine learning (SVMs [2], [3] and Bayesian networks [4], Hidden Markov Models [5]), and, more recently, deep learning [6].

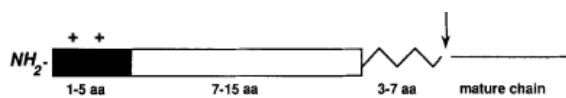


Figure 1: Signal peptide structure

With this work, we propose a comparative analysis of two different approaches for the detection of SPs, namely the Von-Heijne model a Support Vector Machine (SVM) based method.

2. Materials and Methods

2.1. Data Collection

Reviewed non-fragment protein sequences from the eukaryota taxa were queried from UniprotKB, and collected into 2 distinct sets, positive and negative. For Positive sequences, we selected sequences having experimental evidence for the presence of a signal peptide (ft_signal_exp:*) For the negative set, proteins not annotated with a signal peptide and endowed with experimental evidence for subcellular localization in non-secretory compartments, namely nucleus (cc_scl_term_exp:SL-0191), peroxisome (cc_scl_term_exp:SL-0204), cell membrane / plasma membrane (cc_scl_term_exp:SL-0039), cytosol (cc_scl_term_exp:SL-0091), plastid (cc_scl_term_exp:SL-0209) or mitochondrion (cc_scl_term_exp:SL-0173). For sequences in the negative set, we gathered information about the presence of a transmembrane (TM) helix, whose hydrophobicity profile mimicks the one of a signal peptide.

Further selection on the positive entries was carried out in order to select proteins which had information about the cleavage site and a Signal Peptide with a length of at least 14bp.

To perform an initial screening for possible differences between the two sets, we plotted the distribution of sequence lengths in the positive and negative datasets (Figure 2), which showed similar trends. To ensure equal distribution among the kindoms, the composition of each set was considered (Figure 3). The distribution of cleavage-site positions, corresponding to the length of the cleaved N-terminal region in positive sequences, is shown in Figure 4. The distribution was used to quickly scan for outliers and to crudely confirm the biological plausibility of the information.

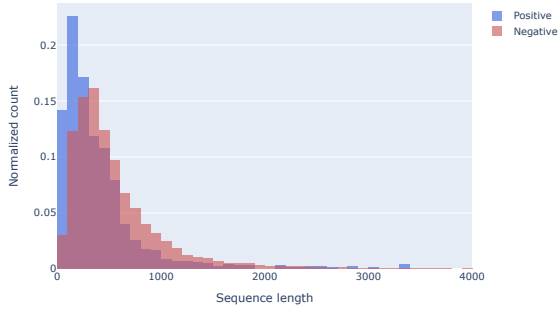


Figure 2: Length distribution of positive and negative sequences

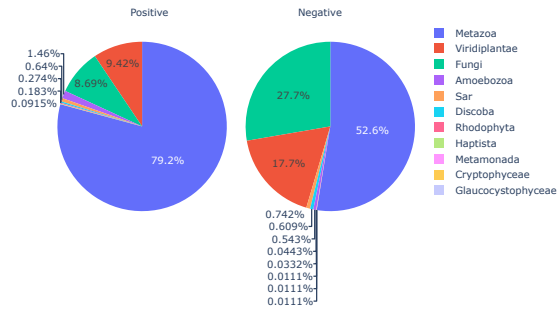


Figure 3: Kingdom distribution of positive and negative datasets

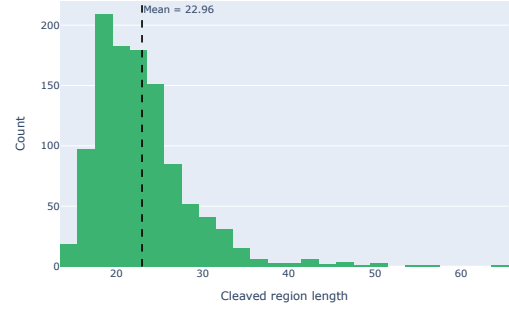


Figure 4: Length distribution of the cleaved region in positive sequences

2.2. Clustering

Sequences were clustered using mmseqs2 [7], using parameters `--min-seq-id 0.3, cluster-mode 1 -cov-mode 0 -c 0.4`, in order to cluster sequences with a similarity of at least 30% and an alignment spanning at least 40% of the total sequence length. The number of sequences in the datasets is reported in Table 1.

Dataset	Before Clustering	After Clustering
Positive	2942	1093
Negative (total)	20806	9028
Negative (w/o TM)	18301	8113
Negative (with TM)	2505	915

Table 1: Dataset sizes before and after clustering

2.3. Von-Heijne Model

The full dataset was split into a training and a test set with a ratio of 0.8. A Position-Specific Weight Matrix (PSWM) was then generated using 16bp windows (13bp upstream to 2bp downstream) surrounding the cleavage site. Amino acid frequencies were normalized using background amino acid frequencies from SwissProt. After initializing the matrix with pseudocounts of 1, the weights are computed as follows:

$$w(a, i) = \log_2 \left(\frac{f(a, i)}{f(a)} \right)$$

Where $f(a, i)$ is the frequency of amino acid a at position i in the training set, and $f(a)$ is the

background frequency of amino acid a in SwissProt. The score for a given sequence is then computed as:

$$S = \max_{j=1}^{90-16+1} \sum_{i=1}^{16} w(a_{j+i-1}, i)$$

scores are computed over the first 90bp of the given sequence, using a sliding window 16bp long. Then, a sequence was labeled as positively endowed with a SP if the Score S is greater than a threshold t .

$$\text{label} = \begin{cases} \text{positive} & \text{if } S > t \\ \text{negative} & \text{if } S \leq t \end{cases}$$

The optimal threshold for the dataset was computed using 5-fold cross validation. For each fold, the data was split into validation and test sets. On the validation set, we computed precision and recall values across all possible threshold values, then selected the threshold that maximized the F1 score (see Section 2.6). This optimal threshold was then evaluated on the corresponding test set to compute performance metrics. The final threshold reported is the average of the optimal thresholds across all 5 folds.

2.4. Feature Extraction

To build a model that can deal with sequences, they must be encoded into numerical representation. There are many possibility of extracting meaningful numerical features from a sequence, for the SVM and MLP model, we employed 2 kinds of features:

- Aminoacid composition up to a cut-off k (n_term_composition) and from k to an hardcoded limit N (set to 90) (c_term_composition)
- Protein Scales computed on the first N aminoacids with a window size of 5 (Figure 5). An Hydrophobicity scale (cit), alpha-helix tendency scale (cit), transmembrane tendency (cit), bulkiness (cit), and polarity (cit) were used.

For this experiment, k was chosen to be the average position of the cut site, roughly 22 aminoacids from the N-Terminus.

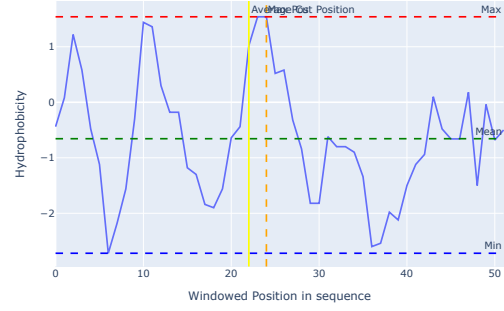


Figure 5: Example of feature extraction for a given sequence. Protein scales are computed on the first N aminoacids with a window size of 5

2.5. Support Vector Machine

Support Vector Machines (SVM) are a machine learning methodology that can be used for two-group classification problems. SVMs separate classes using an hyperplane, defined by a small set of points called support vectors. Albeit trivial for linearly separable classes, vector machines can make use of a kernel function, which allows the mapping of input points from the original space to an higher-dimension space (feature space). In this new space, a linear decision boundary can be established, which, when mapped back to the original space, will produce a complex non-linear decision boundary, enabling correct classification of complex classes [8]. A commonly used kernel function is Radial Basis Function (RBF), which is defined as follows:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

The γ parameter defines how much influence a single training example has. A low γ value means that the influence of a single training example is far-reaching, while a high γ value means that the influence is limited to close neighbors. For both linear and RBF kernels, the C parameter controls the trade-off between maximizing the margin and minimizing the classification error, with low C values leading to a wide margin and a simpler model, and high C values leading to a narrower margin, with few training errors but a more complex model.

We used a grid search to find optimal hyperparameters, testing both RBF and linear kernels. For the RBF kernel, we evaluated C values of 0.1, 1, 2, 4, and 8, combined with gamma values of “scale”,

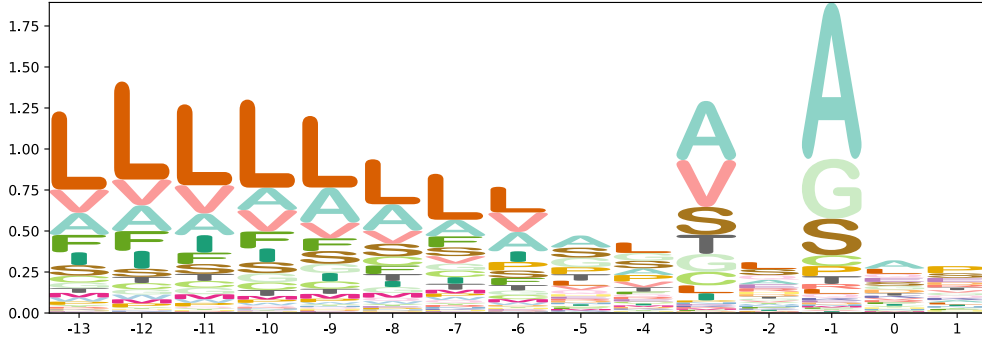


Figure 6: Motif logo for the context (-13aa to 2aa) surrounding the cleavage site

0.001, 0.01, 0.1, 1, and 2. For the linear kernel, we tested C values of 0.1, 1, 2, 4, and 8. Each model was tested on a and evaluated on an 80% training split. The model yielding the highest f1 score was found to have a rbf kernel, with $C = 2$ and $\gamma = 0.01$, with a mean f1 score across the 5 folds ran on the training set of 0.87.

2.6. Evaluation and comparison

Rather than relying on accuracy alone, since the dataset is greatly imbalanced, to evaluate and compare the performance of the models, we computed several metrics on the test set. The metrics were calculated based on the following formulas:

Accuracy: The proportion of correctly classified instances among all instances.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision: The proportion of true positive predictions among all positive predictions.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (Sensitivity): The proportion of true positive instances that were correctly identified.

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score: The harmonic mean of precision and recall.

$$F_1 = 2 \cdot \frac{(\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

Matthews Correlation Coefficient (MCC): A balanced measure that takes into account all four confusion matrix categories.

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Where TP (True Positives), TN (True Negatives), FP (False Positives), and FN (False Negatives) are the values from the confusion matrix.

3. Results

3.1. SP Motifs

From all SP-Endowed sequences, a motif logo (Figure 6) was generated using the context around the cleavage site (13 aa upstream to 2 aa downstream) .

3.2. Von Heijne Model

For the Von-Heijne model the computed optimal threshold was 9.25, confusion matrix computed on the test set is reported in Figure 7. The model achieved an accuracy of 0.9378, precision of 0.7474, recall of 0.6532, F1 score of 0.6971, and MCC of 0.6645. The model thus reaches an high accuracy, but performance on the positive class is limited, detecting only around 65% of actual positives.

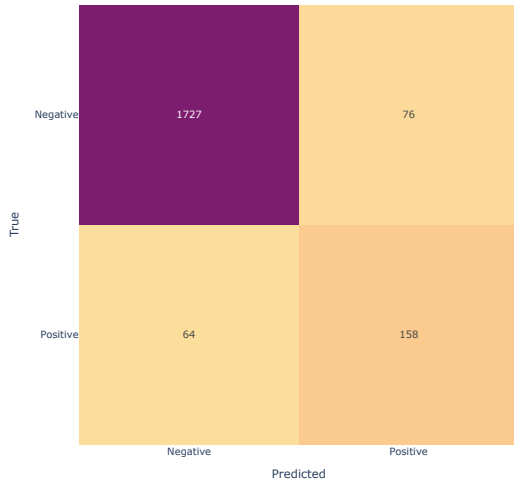


Figure 7: Confusion matrix for the Von-Heijne model

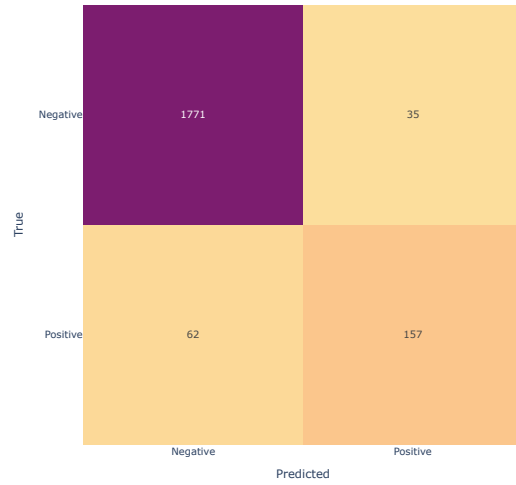


Figure 8: Confusion matrix for the reduced SVM model

3.3. Support Vector Machine

The model, as chosen in the hyperparameter tuning phase, is a SVM with RBF kernel, $C = 2$ and $\gamma = 0.01$. The confusion matrix computed on the test set is reported in Figure 9. The model achieved an accuracy of 0.9743, precision of 0.9034, recall of 0.8539, F1 score of 0.8779, and MCC of 0.8640. The SVM model thus outperforms the Von-Heijne model across all metrics, showing particularly strong improvement in recall (85.39% vs 65.32%), indicating better detection of signal peptides. This means that the SVM model is both more reliable when predicting positives and much better at detecting actual positive cases. It is worth noting that this increase in performance mirrors a great increase in complexity of the underlying method, which is structurally more complex than a simple PSWM, and also requires far greater preprocessing to compute the features. To mitigate this, we tried producing a reduced model, using only the top 5 features selected using permutation importance.

3.3.1. Feature Importance

In order to understand which features contribute the most to the ability of our model to classify sequences, we employed two methods. As a first test, we assessed permutation importance, where values of each feature are randomly shuffled in order to break their relationship with the target label, using the drop in the classifier accuracy as a proxy for the importance of the feature. As a second test, we trained a Random Forest Classifier with 500 classifiers and exploited the model's own feature importance, which is computed as a result of the training process. We also evaluated the performance of the resulting random forest classifier, which itself proved to be a good model, with an accuracy of 0.9580, precision of 0.9036, recall of 0.6849, F1 score of 0.7792, and MCC of 0.7654.

The top 5 features identified by each method are shown in Table 2 and Table 3. Both methods consistently identified `n_terminal_comp_L` (leucine composition in the N-terminal region) and `tm_max` (maximum transmembrane tendency) as important features, even if with different ranks.

Feature	Importance Mean	Importance Std
n_terminal_comp_D	0.068479	0.002761
n_terminal_comp_A	0.010622	0.001014
tm_max	0.008453	0.001321
kd_max_pos	0.008225	0.001022
n_terminal_comp_L	0.008195	0.001233

Table 2: Top 5 features by permutation importance

Feature	Importance
kd_max_pos	0.11984
tm_max_pos	0.105942
tm_max	0.091583
kd_max	0.078178
n_terminal_comp_L	0.06908

Table 3: Top 5 features by random forest importance

3.3.2. Reduced SVM Model

We then trained a reduced model, using the combination of 5 most important features found by each method. The chosen features were:

- kd_max: Maximum hydrophobicity value
- kd_max_pos: Position of maximum hydrophobicity
- n_terminal_comp_A: Alanine composition in the N-terminal region
- n_terminal_comp_D: Aspartic acid composition in the N-terminal region
- n_terminal_comp_L: Leucine composition in the N-terminal region
- tm_max: Maximum transmembrane tendency
- tm_max_pos: Position of maximum transmembrane tendency

A model was selected using the same method used to select the full model, including the same pool of possible classifiers. The reduced model achieved an accuracy of 0.9521, precision of 0.8177, recall of 0.7169, F1 score of 0.7640, and MCC of 0.7394. The confusion matrix computed on the test set is reported in Figure 8. While this represents a decrease in performance compared to the full SVM model (F1 score of 0.8779 vs 0.7640), the reduced model still outperforms the Von-Heijne model (F1 score of 0.6971) while using only 7 features instead of the full feature set.

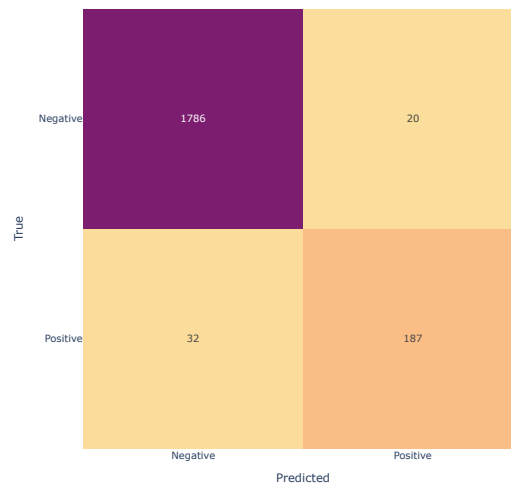


Figure 9: Confusion matrix for the SVM model

Bibliography

- [1] G. von Heijne, "The signal peptide," *The Journal of Membrane Biology*, vol. 115, no. 3, pp. 195–201, May 1990, doi: 10.1007/BF01868635.
- [2] T. Nugent and D. T. Jones, "Transmembrane protein topology prediction using support vector machines," *BMC bioinformatics*, vol. 10, p. 159, May 2009, doi: 10.1186/1471-2105-10-159.
- [3] T. N. Petersen, S. Brunak, G. von Heijne, and H. Nielsen, "SignalP 4.0: discriminating signal peptides from transmembrane regions," *Nature Methods*, vol. 8, no. 10, pp. 785–786, Sept. 2011, doi: 10.1038/nmeth.1701.
- [4] S. M. Reynolds, L. Käll, M. E. Riffle, J. A. Bilmes, and W. S. Noble, "Transmembrane topology and signal peptide prediction using dynamic bayesian networks," *PLoS computational biology*, vol. 4, no. 11, p. e1000213, Nov. 2008, doi: 10.1371/journal.pcbi.1000213.
- [5] L. Käll, A. Krogh, and E. L. Sonnhammer, "A Combined Transmembrane Topology and Signal Peptide Prediction Method," *Journal of Molecular Biology*, vol. 338, no. 5, pp. 1027–1036, May 2004, doi: 10.1016/j.jmb.2004.03.016.

- [6] C. Savojardo, P. L. Martelli, P. Fariselli, and R. Casadio, “DeepSig: deep learning improves signal peptide detection in proteins,” *Bioinformatics*, vol. 34, no. 10, pp. 1690–1696, May 2018, doi: 10.1093/bioinformatics/btx818.
- [7] M. Steinegger and J. Söding, “MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets,” *Nature Biotechnology*, vol. 35, no. 11, pp. 1026–1028, Nov. 2017, doi: 10.1038/nbt.3988.
- [8] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, Sept. 1995, doi: 10.1007/BF00994018.